



Freie Universität Bozen
Libera Università di Bolzano
Università Lìedia de Bulsan



Birkbeck
UNIVERSITY OF LONDON



PADERBORN
UNIVERSITY

Querying Interval-Based Temporal Data with SPARQL

33rd International Symposium on Temporal Representation and Reasoning

September 1st 2026 | Cork, Ireland

Julien Corman

Roman Kontchakov

Cem Okulmus

Free University of Bolzano, Italy

Birkbeck, University of London, UK

Paderborn University, Germany



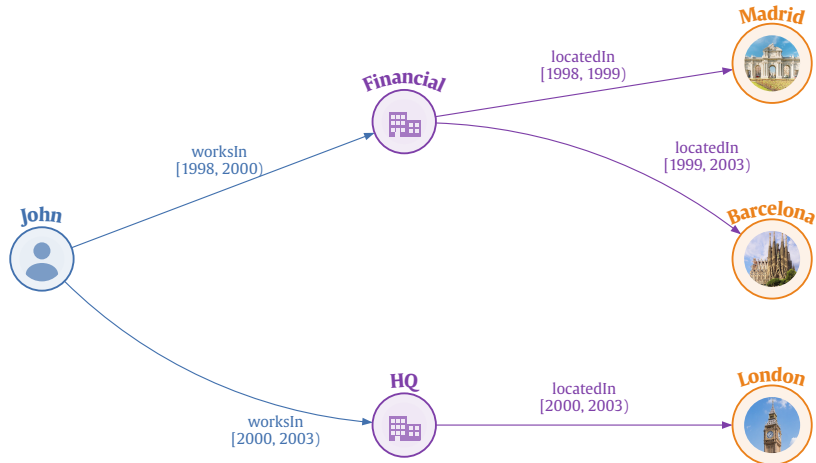
- Knowledge Graphs (KGs) see increasing use:
 - Can cover missing, ill-typed and/or heterogeneous data

Motivation

- Knowledge Graphs (KGs) see increasing use:
 - Can cover missing, ill-typed and/or heterogeneous data
- More and more interest in **temporal data inside KGs**

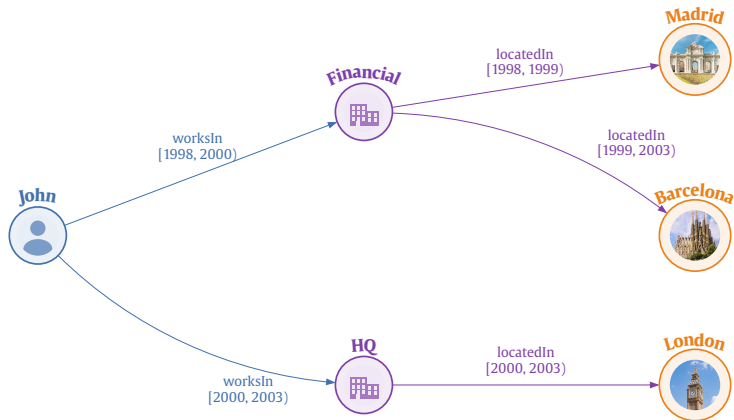
- Knowledge Graphs (KGs) see increasing use:
 - Can cover missing, ill-typed and/or heterogeneous data
- More and more interest in **temporal data inside KGs**
- Need for Query Languages that can express **temporal queries**:
 - Easily expressing temporal joins
 - Comparisons across intervals in time

Example: Temporal KG



Querying temporal KGs can produce new intervals

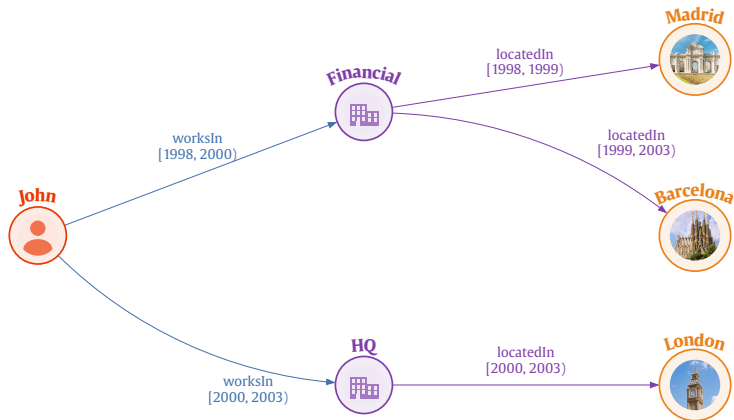
“Employees Query”:
Which employees at **a given interval** work in a department located in Barcelona, which **previously** was located in Madrid?



Querying temporal KGs can produce new intervals

“Employees Query”:
Which employees at **a given interval** work in a department located in Barcelona, which **previously** was located in Madrid?

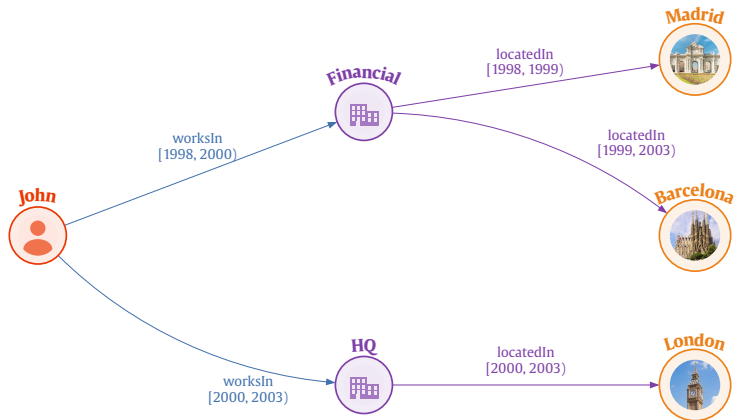
- Answer:
{ John }



Querying temporal KGs can produce new intervals

“Employees Query”:
Which employees at **a given interval** work in a department located in Barcelona, which **previously** was located in Madrid?

- Answer:
{ John@[1999,2000) }



We present **Temporal SPARQL**

Employees Query, rendered in Temporal SPARQL

```
PREFIX :      <http://www.example.org#>
PREFIX time:  <http://www.w3.org/2006/time#>

SELECT ?e ?dep1 ?t1 {
  << ?e      :worksIn    ?dep1      >> time:hasTime ?t1.
  << ?dep1   :locatedIn  :barcelona >> time:hasTime ?t1.
  FILTER EXISTS {
    << ?dep1 :locatedIn :madrid >> time:hasTime ?t2.
    FILTER ( time:Before(?t2,?t1) ).
  }
}
```

“Employees Query”:

Which employees at a given interval work in a department located in Barcelona, which previously was located in Madrid?

We present **Temporal SPARQL**

- Making use of SPARQL 1.2 and its *annotated triples* to attach intervals to facts

Employees Query, rendered in Temporal SPARQL

```
PREFIX :      <http://www.example.org#>
PREFIX time:  <http://www.w3.org/2006/time#>

SELECT ?e ?dep1 ?t1 {
  << ?e      :worksIn      ?dep1      >> time:hasTime ?t1.
  << ?dep1    :locatedIn    :barcelona >> time:hasTime ?t1.
  FILTER EXISTS {
    << ?dep1    :locatedIn    :madrid    >> time:hasTime ?t2.
    FILTER ( time:Before(?t2,?t1) ).
  }
}
```

“Employees Query”:

Which employees at a given interval work in a department located in Barcelona, which previously was located in Madrid?

We present **Temporal SPARQL**

- Making use of SPARQL 1.2 and its *annotated triples* to attach intervals to facts
- Use of *temporal variables* to easily express implicit joins

Employees Query, rendered in Temporal SPARQL

```
PREFIX : <http://www.example.org#>
PREFIX time: <http://www.w3.org/2006/time#>

SELECT ?e ?dep1 ?t1 {
  << ?e :worksIn ?dep1 >> time:hasTime ?t1.
  << ?dep1 :locatedIn :barcelona >> time:hasTime ?t1.
  FILTER EXISTS {
    << ?dep1 :locatedIn :madrid >> time:hasTime ?t2.
    FILTER ( time:Before(?t2,?t1) ).
  }
}
```

“Employees Query”:

Which employees at a given interval work in a department located in Barcelona, which previously was located in Madrid?

We present **Temporal SPARQL**

- Making use of SPARQL 1.2 and its *annotated triples* to attach intervals to facts
- Use of *temporal variables* to easily express implicit joins
- Support for *temporal predicates*, allowing to express Allen's relations over interval variables

Employees Query, rendered in Temporal SPARQL

```
PREFIX : <http://www.example.org#>
PREFIX time: <http://www.w3.org/2006/time#>

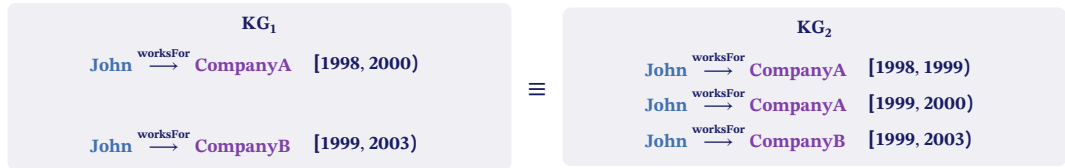
SELECT ?e ?dep1 ?t1 {
  << ?e :worksIn ?dep1 >> time:hasTime ?t1.
  << ?dep1 :locatedIn :barcelona >> time:hasTime ?t1.
  FILTER EXISTS {
    << ?dep1 :locatedIn :madrid >> time:hasTime ?t2.
    FILTER ( time:Before(?t2,?t1) ).
  }
}
```

“Employees Query”:

Which employees at a given interval work in a department located in Barcelona, which previously was located in Madrid?

Challenges

Challenge 1: We can only compare coalesced intervals



Challenge 1: We can only compare coalesced intervals

KG₁

John ^{worksFor} → CompanyA [1998, 2000)

John ^{worksFor} → CompanyB [1999, 2003)

≡

KG₂

John ^{worksFor} → CompanyA [1998, 1999)

John ^{worksFor} → CompanyA [1999, 2000)

John ^{worksFor} → CompanyB [1999, 2003)

*Which employees of CompanyA
have worked previously for
CompanyB?*

Temporal SPARQL Query Q

```
SELECT ?e ?t2 {
  << ?e :worksFor :CompanyA >> time:hasTime ?t1.
  FILTER EXISTS {
    << ?e :worksFor :CompanyB >> time:hasTime ?t2.
    FILTER ( time:Before(?t2,?t1) ).
  }
}
```

Challenge 1: We can only compare coalesced intervals

KG₁

John ^{worksFor} → CompanyA [1998, 2000)

John ^{worksFor} → CompanyB [1999, 2003)

≡

KG₂

John ^{worksFor} → CompanyA [1998, 1999)

John ^{worksFor} → CompanyA [1999, 2000)

John ^{worksFor} → CompanyB [1999, 2003)

*Which employees of CompanyA
have worked previously for
CompanyB?*

Temporal SPARQL Query Q

```
SELECT ?e ?t2 {
  << ?e :worksFor :CompanyA >> time:hasTime ?t1.
  FILTER EXISTS {
    << ?e :worksFor :CompanyB >> time:hasTime ?t2.
    FILTER ( time:Before(?t2,?t1) ).
  }
}
```

$Q(KG_1) = \emptyset$, however $Q(KG_2) = \{ \text{John}@[1999,2003) \}$ **Why?**

Challenge 1: We can only compare coalesced intervals

KG₁

John $\xrightarrow{\text{worksFor}}$ CompanyA [1998, 2000)

John $\xrightarrow{\text{worksFor}}$ CompanyB [1999, 2003)

≡

KG₂

John $\xrightarrow{\text{worksFor}}$ CompanyA [1998, 1999)

John $\xrightarrow{\text{worksFor}}$ CompanyA [1999, 2000)

John $\xrightarrow{\text{worksFor}}$ CompanyB [1999, 2003)

*Which employees of CompanyA
have worked previously for
CompanyB?*

Temporal SPARQL Query Q

```
SELECT ?e ?t2 {
  << ?e :worksFor :CompanyA >> time:hasTime ?t1.
  FILTER EXISTS {
    << ?e :worksFor :CompanyB >> time:hasTime ?t2.
    FILTER ( time:Before(?t2,?t1) ).
  }
}
```

$Q(KG_1) = \emptyset$, however $Q(KG_2) = \{ \text{John}@[1999,2003) \}$! **Why?**

⇒ We need **normalised, i.e. coalesced, temporal KGs!**

Challenge 2: SPARQL is not SQL

- Certain operations are known not to preserve coalescing, e.g. union or projection
- These require an explicit coalescing operation to be performed afterwards

Challenge 2: SPARQL is not SQL

- Certain operations are known not to preserve coalescing, e.g. union or projection
- These require an explicit coalescing operation to be performed afterwards

⇒ In temporal relational algebra, *inner join* preserves coalescing¹

¹Böhlen, Snodgrass & Soo, *Coalescing in Temporal Databases*, VLDB'96

Challenge 2: SPARQL is not SQL

- Certain operations are known not to preserve coalescing, e.g. union or projection
- These require an explicit coalescing operation to be performed afterwards
 - ⇒ In temporal relational algebra, *inner join* preserves coalescing¹
 - ⇒ In SPARQL it does **not** — our semantics has to coalesce *after* a join

¹Böhlen, Snodgrass & Soo, *Coalescing in Temporal Databases*, VLDB'96

Challenge 2: SPARQL is not SQL

- Certain operations are known not to preserve coalescing, e.g. union or projection
- These require an explicit coalescing operation to be performed afterwards

⇒ In temporal relational algebra, *inner join* preserves coalescing¹

⇒ In SPARQL it does **not** – our semantics has to coalesce *after* a join

- SPARQL semantics uses **solution mappings**

▪ Sets of *partial* functions $\overset{\text{Variables}}{V} \rightarrow \overset{\text{Domain Objects}}{D}$

¹Böhlen, Snodgrass & Soo, *Coalescing in Temporal Databases*, VLDB'96

Challenge 2: SPARQL is not SQL

- Certain operations are known not to preserve coalescing, e.g. union or projection
- These require an explicit coalescing operation to be performed afterwards
 - ⇒ In temporal relational algebra, *inner join* preserves coalescing¹
 - ⇒ In SPARQL it does **not** – our semantics has to coalesce *after* a join
- SPARQL semantics uses **solution mappings**
 - Sets of *partial* functions $\overset{\text{Variables}}{V} \rightarrow \overset{\text{Domain Objects}}{D}$
- SQL instead works on tuples over a fixed schema, i.e. sets of total functions

¹Böhlen, Snodgrass & Soo, *Coalescing in Temporal Databases*, VLDB'96

- Certain operations are known not to preserve coalescing, e.g. union or projection
- These require an explicit coalescing operation to be performed afterwards

⇒ In temporal relational algebra, *inner join* preserves coalescing¹

⇒ In SPARQL it does **not** – our semantics has to coalesce *after* a join

- SPARQL semantics uses **solution mappings**

▪ Sets of *partial* functions $\overset{\text{Variables}}{V} \rightarrow \overset{\text{Domain Objects}}{D}$

- SQL instead works on tuples over a fixed schema, i.e. sets of total functions

SPARQL joins do not preserve coalescing

$$S_1 = \left\{ ([0, 1), \{x \mapsto :a\}) \right. \\ \left. ([1, 2), \{y \mapsto :b\}) \right\} \quad S_2 = \{ ([0, 2), \{x \mapsto :a, y \mapsto :b\}) \}$$
$$S_1 \bowtie S_2 = \left\{ ([0, 1), \{x \mapsto :a, y \mapsto :b\}) \right. \\ \left. ([1, 2), \{x \mapsto :a, y \mapsto :b\}) \right\}$$

¹Böhlen, Snodgrass & Soo, *Coalescing in Temporal Databases*, VLDB'96

Challenge 3: Multiple Time Variables Lead To Intractability

Consider a variation of the prior Q :

Temporal SPARQL Query Q'

```
SELECT ?e ?t1 ?t2 {  
  << ?e :worksIn :Financial >> time:hasTime ?t1.  
  << ?e :worksIn :HQ >> time:hasTime ?t2.  
  FILTER ( time:Before(?t1,?t2) ).  
}
```

How to normalise the output? We now need to coalesce a **2D structure**!

Challenge 3: Multiple Time Variables Lead To Intractability

Consider a variation of the prior Q :

Temporal SPARQL Query Q'

```
SELECT ?e ?t1 ?t2 {  
  << ?e :worksIn :Financial >> time:hasTime ?t1.  
  << ?e :worksIn :HQ >> time:hasTime ?t2.  
  FILTER ( time:Before(?t1,?t2) ).  
}
```

How to normalise the output? We now need to coalesce a **2D structure**!

- Answer is cover of (t_1, t_2) plane

Challenge 3: Multiple Time Variables Lead To Intractability

Consider a variation of the prior Q :

Temporal SPARQL Query Q'

```
SELECT ?e ?t1 ?t2 {  
  << ?e :worksIn :Financial >> time:hasTime ?t1.  
  << ?e :worksIn :HQ >> time:hasTime ?t2.  
  FILTER ( time:Before(?t1,?t2) ).  
}
```

How to normalise the output? We now need to coalesce a **2D structure**!

- Answer is cover of (t_1, t_2) plane

⇒ Deciding whether a minimal (overlapping) set of covers exists is **NP-hard**¹

¹Culberson & Reckhow, *Covering Polygons is hard*, FOCS'88

Challenge 3: Multiple Time Variables Lead To Intractability

Consider a variation of the prior Q :

Temporal SPARQL Query Q'

```
SELECT ?e ?t1 ?t2 {  
  << ?e :worksIn :Financial >> time:hasTime ?t1.  
  << ?e :worksIn :HQ >> time:hasTime ?t2.  
  FILTER ( time:Before(?t1,?t2) ).  
}
```

How to normalise the output? We now need to coalesce a **2D structure**!

- Answer is cover of (t_1, t_2) plane
- ⇒ Deciding whether a minimal (overlapping) set of covers exists is **NP-hard**¹
- ⇒ Restrict use of multiple time variables!

¹Culberson & Reckhow, *Covering Polygons is hard*, FOCS'88

Semantics of Temporal SPARQL

A **snapshot KG** G : one RDF graph $G(\tau)$ per time point τ

Abstract Model

Snapshot View on Time

A **snapshot KG** G : one RDF graph $G(\tau)$ per time point τ

Snapshot semantics

$\llbracket P \rrbracket_s^{G(\tau)} \dots$ plain SPARQL semantics over $G(\tau)$

No new algebra – time only picks the graph.

1998	John	worksIn	Financial
1999	John	worksIn	Financial
2000	John	worksIn	HQ
			⋮

Abstract Model

Snapshot View on Time

A **snapshot KG** G : one RDF graph $G(\tau)$ per time point τ

Snapshot semantics

$\llbracket P \rrbracket_s^{G(\tau)} \dots$ plain SPARQL semantics over $G(\tau)$

No new algebra – time only picks the graph.

1998	John	worksIn	Financial
1999	John	worksIn	Financial
2000	John	worksIn	HQ
			⋮

- Snapshot Semantics cannot compare intervals
- The queries P exclude comparisons of τ objects

Logical Model $\therefore$ Annotation Semirings

Abstract models cannot relate facts to intervals
 $\Rightarrow$ Map each fact to the **set of times** it holds at

An **interval KG** G : maps an interval I to a graph

This forms an SPM-semiring¹

$$\mathcal{S}_{\mathbb{T}} = \langle S_{\mathbb{T}}, \cup, \cap, \setminus, \emptyset, \mathbb{T} \rangle$$

$$\text{Union} = \cup \quad \text{Join} = \cap \quad \text{Minus} = \setminus$$

¹Geerts et al., *Algebraic structures for capturing the provenance of SPARQL queries*, JACM'16

Logical Model $\therefore$ Annotation Semirings

Abstract models cannot relate facts to intervals
 $\Rightarrow$ Map each fact to the **set of times** it holds at

An **interval KG** G : maps an interval I to a graph

Annotation semantics

$\llbracket P \rrbracket_a^G \dots$ outputs pairs of facts and sets of intervals

Can **compare intervals** now

This forms an SPM-semiring¹

$$\mathcal{S}_T = \langle S_T, \cup, \cap, \setminus, \emptyset, T \rangle$$

Union = $\cup$ Join = $\cap$ Minus = $\setminus$

John $\xrightarrow{\text{worksIn}}$ Financial $\{[1998, 2000]\}$
 John $\xrightarrow{\text{worksIn}}$ HQ $\{[2000, 2003], [2006, 2008]\}$
 a set of intervals per fact

¹Geerts et al., *Algebraic structures for capturing the provenance of SPARQL queries*, JACM'16

Logical Model ∴

Annotation Semirings

Abstract models cannot relate facts to intervals
 ⇒ Map each fact to the **set of times** it holds at

An **interval KG** G : maps an interval I to a graph

Annotation semantics

$\llbracket P \rrbracket_a^G \dots$ outputs pairs of facts and sets of intervals

Can **compare intervals** now

For a **reduced fragment**:
 translation between Logical Model and Abstract
 Model is possible!

This forms an SPM-semiring¹

$$\mathcal{S}_T = \langle S_T, \cup, \cap, \setminus, \emptyset, T \rangle$$

Union = $\cup$ Join = $\cap$ Minus = $\setminus$

John $\xrightarrow{\text{worksIn}}$ Financial $\{[1998, 2000]\}$
 John $\xrightarrow{\text{worksIn}}$ HQ $\{[2000, 2003], [2006, 2008]\}$

a set of intervals per fact

¹Geerts et al., *Algebraic structures for capturing the provenance of SPARQL queries*, JACM'16

Logical Model ∴

Annotation Semirings

Abstract models cannot relate facts to intervals
 $\Rightarrow$ Map each fact to the **set of times** it holds at

An **interval KG** G : maps an interval I to a graph

Annotation semantics

$\llbracket P \rrbracket_a^G \dots$ outputs pairs of facts and sets of intervals

Can **compare intervals** now

For a **reduced fragment**:
 translation between Logical Model and Abstract Model is possible!

This forms an SPM-semiring¹

$$\mathcal{S}_T = \langle S_T, \cup, \cap, \setminus, \emptyset, T \rangle$$

Union = $\cup$ Join = $\cap$ Minus = $\setminus$

John $\xrightarrow{\text{worksIn}}$ Financial $\{[1998, 2000]\}$
 John $\xrightarrow{\text{worksIn}}$ HQ $\{[2000, 2003], [2006, 2008]\}$

a set of intervals per fact

Snapshot-reducibility

$(\mu, I) \in \llbracket P \rrbracket_a^G$ iff $\exists I \in I. \exists \tau \in I. (\mu \in \llbracket P \rrbracket_s^{G(\tau)})$
 for **representable** G and **snapshot-pattern** P .

¹Geerts et al., *Algebraic structures for capturing the provenance of SPARQL queries*, JACM'16

Concrete Model

View on Implementation

Logical Model is hard to implement: unbounded sets of intervals per fact
Desideratum: **one interval** per answer

Concrete Model

View on Implementation

Logical Model is hard to implement: unbounded sets of intervals per fact
 Desideratum: **one interval** per answer

Interval Semantics

$\llbracket P \rrbracket_i^G$... outputs pairs of (fact + interval)

fact	interval
John $\xrightarrow{\text{worksIn}}$ Financial	[1998, 2000)
John $\xrightarrow{\text{worksIn}}$ HQ	[2000, 2003)
John $\xrightarrow{\text{worksIn}}$ HQ	[2006, 2008)

one interval per row

Concrete Model

View on Implementation

Logical Model is hard to implement: unbounded sets of intervals per fact
 Desideratum: **one interval** per answer

Interval Semantics

$\llbracket P \rrbracket_i^G$... outputs pairs of (fact + interval)

fact	interval
John $\xrightarrow{\text{worksIn}}$ Financial	[1998, 2000)
John $\xrightarrow{\text{worksIn}}$ HQ	[2000, 2003)
John $\xrightarrow{\text{worksIn}}$ HQ	[2006, 2008)

one interval per row

- Coalescing becomes an operator, C , instead of a property of the data model

Logical Model is hard to implement: unbounded sets of intervals per fact
Desideratum: **one interval** per answer

Interval Semantics

$\llbracket P \rrbracket_i^G$... outputs pairs of (fact + interval)

fact	interval
John $\xrightarrow{\text{worksIn}}$ Financial	[1998, 2000)
John $\xrightarrow{\text{worksIn}}$ HQ	[2000, 2003)
John $\xrightarrow{\text{worksIn}}$ HQ	[2006, 2008)

one interval per row

- Coalescing becomes an operator, C , instead of a property of the data model

We show the following equivalence between the Logical and Concrete Model

$$\forall P \forall G. (\llbracket P \rrbracket_i^G = C(\llbracket P \rrbracket_a^G))$$

Overview of The Three Semantics

Abstract Model

a graph per instant

1998 John $\xrightarrow{\text{worksIn}}$ Financial
1999 John $\xrightarrow{\text{worksIn}}$ Financial
2000 John $\xrightarrow{\text{worksIn}}$ HQ
⋮

Overview of The Three Semantics

Abstract Model

a graph per instant

1998 John $\xrightarrow{\text{worksIn}}$ Financial
1999 John $\xrightarrow{\text{worksIn}}$ Financial
2000 John $\xrightarrow{\text{worksIn}}$ HQ
⋮

Logical Model $\therefore$

a set of intervals per fact

John $\xrightarrow{\text{worksIn}}$ Fin. $\{\{1998, 2000\}\}$
John $\xrightarrow{\text{worksIn}}$ HQ $\{\{2000, 2003\},$
 $\quad\quad\quad [2006, 2008]\}$

Abstract Model

a graph per instant

1998 John $\xrightarrow{\text{worksIn}}$ Financial
1999 John $\xrightarrow{\text{worksIn}}$ Financial
2000 John $\xrightarrow{\text{worksIn}}$ HQ
⋮

Logical Model $\therefore$

a set of intervals per fact

John $\xrightarrow{\text{worksIn}}$ Fin. $\{[1998, 2000]\}$
John $\xrightarrow{\text{worksIn}}$ HQ $\{[2000, 2003],$
 $[2006, 2008]\}$

Concrete Model

one interval per row

fact	period
John $\xrightarrow{\text{worksIn}}$ Financial	[1998, 2000)
John $\xrightarrow{\text{worksIn}}$ HQ	[2000, 2003)
John $\xrightarrow{\text{worksIn}}$ HQ	[2006, 2008)

Abstract Model

a graph per instant

1998	John	worksIn	Financial
1999	John	worksIn	Financial
2000	John	worksIn	HQ
⋮			

collect



eval. at τ

Logical Model $\therefore$

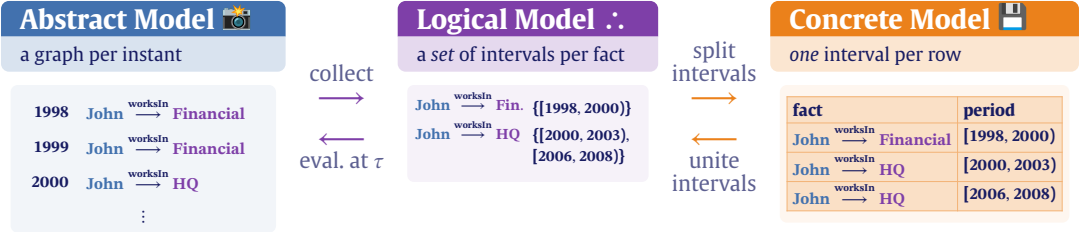
a set of intervals per fact

John	worksIn	Fin.	{[1998, 2000]}
John	worksIn	HQ	{[2000, 2003], [2006, 2008]}

Concrete Model

one interval per row

fact	period
John worksIn Financial	[1998, 2000)
John worksIn HQ	[2000, 2003)
John worksIn HQ	[2006, 2008)



Temporal SPARQL and Ontology-Based Data Access

Ontology-based Data Access (OBDA)

Collection of techniques to extend (relational) data with ontological axioms and enabling efficient querying over the extended signature.

Temporal SPARQL and Ontology-Based Data Access

Ontology-based Data Access (OBDA)

Collection of techniques to extend (relational) data with ontological axioms and enabling efficient querying over the extended signature.

Initial motivation for Temporal SPARQL:

extend the OBDA system Ontop, which uses SPARQL, to temporal setting

Temporal SPARQL and Ontology-Based Data Access

Ontology-based Data Access (OBDA)

Collection of techniques to extend (relational) data with ontological axioms and enabling efficient querying over the extended signature.

Initial motivation for Temporal SPARQL:

extend the OBDA system Ontop, which uses SPARQL, to temporal setting

Small Experimental Study to showcase feasibility

- Focus on example temporal query from¹
- Needed **45s** over Wikidata endpoint, but **under 1s** when translated to SQL

Married Athletes Query

```
SELECT DISTINCT ?x ?z ?c ?t
WHERE {
  << ?x :playsFor ?c >> time:hasTime ?t .
  << ?x :marriedTo ?z >> time:hasTime ?t .
}
```

¹Chekol, Pirrò and Stuckenschmidt, *Fast interval joins for temporal SPARQL queries*, WWW'19

Conclusion and Outlook

- **Temporal SPARQL:** supports temporal variables and Allen's relations
+ structural restrictions to ensure tractability

Conclusion and Outlook

- **Temporal SPARQL:** supports temporal variables and Allen's relations + structural restrictions to ensure tractability
- **Three semantics,** tied together by two translations

Conclusion and Outlook

- **Temporal SPARQL:** supports temporal variables and Allen's relations + structural restrictions to ensure tractability
- **Three semantics,** tied together by two translations
- **Feasible:** the concrete model translates to SQL, and beats naive translations to SPARQL over triple stores

Conclusion and Outlook

- **Temporal SPARQL:** supports temporal variables and Allen's relations + structural restrictions to ensure tractability
- **Three semantics**, tied together by two translations
- **Feasible:** the concrete model translates to SQL, and beats naive translations to SPARQL over triple stores

Future Work

- Tractable extensions of Temporal SPARQL: aggregation over time
- Exploring options to reduce need for coalescing
- Using Temporal SPARQL for Temporal OBDA

Thanks for your attention!